

Healthcare Track -- Prompt Library

All prompts are copy-paste ready. Each prompt has a primary version and a fallback.

Step 1: Database Setup

1.1 Primary Prompt

```
Please execute the setup script from the GitHub repository we connected.
Run this file to create the healthcare database, warehouse, all tables,
internal stages, and load all sample data:

@HOL_UTILS.PUBLIC.SNOWFLAKE_AI_HOL_REPO/branches/main/healthcare/scripts/setup.sql

Use EXECUTE IMMEDIATE FROM to run it. After execution, set
HEALTHCARE_HOL_WH as the active warehouse for the rest of our session
and give me a brief summary of what was created.
```

1.1 Fallback Prompt

```
Please read the contents of the file at this Git stage path and execute
it as a SQL script, running each statement sequentially:

@HOL_UTILS.PUBLIC.SNOWFLAKE_AI_HOL_REPO/branches/main/healthcare/scripts/setup.sql

Break it into logical sections and show me progress after each section.
```

1.2 Inspect Prompt

```
Show me all the tables in HEALTHCARE_HOL_DB.CLINICAL and
HEALTHCARE_HOL_DB.OPERATIONS with their row counts. Display as a
clean summary table.
```

Step 2: Semantic Models

2.1 Primary Prompt

```
Create a Cortex Analyst semantic view for the clinical data in
HEALTHCARE_HOL_DB.CLINICAL.

Include these tables: PATIENTS, PROVIDERS, DEPARTMENTS, ENCOUNTERS,
DIAGNOSES, PROCEDURES, MEDICATIONS, LAB_RESULTS, BEDS, and READMISSIONS.

Business intelligence rules:
- Flag "high readmission risk" when RISK_SCORE > 0.7
```

- Flag "extended stay" when LENGTH_OF_STAY > 7
- Flag "critical lab" when RESULT_FLAG = 'Critical'

Make sure it can answer questions like:

1. "How many patients are currently admitted?"
2. "What is the average length of stay by department?"
3. "Which patients are at high risk for readmission?"
4. "Show me all critical lab results"

Name it PATIENT_CARE and store it in HEALTHCARE_HOL_DB.CLINICAL.

Register it directly as a semantic view without saving any intermediate files to a stage.

2.1 Fallback Prompt

Copy the pre-built patient care semantic model from the GitHub repository into the semantic models stage:

Source:

@HOL_UTILS.PUBLIC.SNOWFLAKE_AI_HOL_REPO/branches/main/healthcare/scripts/semantic_models/PATIENT_CARE_MODEL.yaml

Destination: @HEALTHCARE_HOL_DB.CLINICAL.SEMANTIC_MODELS_STAGE

Use COPY FILES INTO to transfer it.

2.2 Claims Model

Create a Cortex Analyst semantic view for insurance claims analytics.

Include these tables:

- HEALTHCARE_HOL_DB.OPERATIONS.INSURANCE_CLAIMS
- HEALTHCARE_HOL_DB.CLINICAL.PATIENTS
- HEALTHCARE_HOL_DB.CLINICAL.ENCOUNTERS

Join INSURANCE_CLAIMS to PATIENTS on PATIENT_ID, and to ENCOUNTERS on ENCOUNTER_ID.

Business intelligence rules:

- Flag "claim denied" when STATUS = 'Denied'
- Calculate approval rate as percentage of claims with STATUS = 'Approved'
- Calculate average reimbursement as APPROVED_AMOUNT / CLAIM_AMOUNT

Make sure it can answer questions like:

1. "What is the total claim amount by insurance type?"
2. "Which claims were denied?"
3. "What is the approval rate by department?"
4. "Show average reimbursement percentage by insurance type"

Name it CLAIMS_ANALYTICS and store it in HEALTHCARE_HOL_DB.CLINICAL.

Register it directly as a semantic view without saving any intermediate files to a stage.

2.3 Explore -- Snowsight UI

Guide attendees to Cortex > Analyst. Select HEALTHCARE_HOL_DB > CLINICAL > PATIENT_CARE. Ask a question, then click "Add to Verified Queries". Walk through the semantic view tour: Custom Instructions, Logical Tables, Relationships, Verified Queries, Suggestions.

Step 3: Cortex Search

3.1 Bring PDF into Snowflake

Bring the following PDF from our GitHub repository into HEALTHCARE_HOL_DB.CLINICAL so we can make it searchable:

@HOL_UTILS.PUBLIC.SNOWFLAKE_AI_HOL_REPO/branches/main/healthcare/pdfs/Clinical Operations Manual.pdf

3.2 Primary Prompt

Create a Cortex Search service called CLINICAL_DOCS_INFO in HEALTHCARE_HOL_DB.CLINICAL that makes the clinical operations PDF searchable with natural language.

To do this:

- Temporarily scale up HEALTHCARE_HOL_WH for processing power
- Read and parse the PDF content
- Break it into overlapping chunks for better search accuracy
- Index everything so it can be queried in plain English
- Scale the warehouse back down to SMALL when done

3.3 Test Search -- Snowsight UI

Guide attendees to AI & ML > Search to test CLINICAL_DOCS_INFO.

Step 4: Intelligence Agent

4.1 Primary Prompt

Create a Snowflake Intelligence Agent named HEALTHCARE_AGENT in HEALTHCARE_HOL_DB.CLINICAL.

Give it three tools:

1. A data analysis tool named PATIENT_CARE_DATA connected to the PATIENT_CARE semantic model in HEALTHCARE_HOL_DB.CLINICAL. It answers questions about patients, encounters, diagnoses, labs, and bed management. Use HEALTHCARE_HOL_WH.
2. A data analysis tool named CLAIMS_DATA connected to the CLAIMS_ANALYTICS semantic model in HEALTHCARE_HOL_DB.CLINICAL. It analyzes insurance claims, approval rates, and billing data. Use HEALTHCARE_HOL_WH.
3. A document search tool named CLINICAL_DOCS connected to the CLINICAL_DOCS_INFO search service in HEALTHCARE_HOL_DB.CLINICAL. It searches clinical documentation for protocols and procedures.

4.2/4.3 -- Snowsight UI

Step 5: Streamlit App

5.1 Chat Page

Build a single-page Streamlit app in Snowflake called HEALTHCARE_ASSISTANT_APP in HEALTHCARE_HOL_DB.CLINICAL using HEALTHCARE_HOL_WH.

Use `st.tabs()` to create a horizontal tab bar at the top of the page with three tabs: "Chat", "Dashboard", and "Optimization". All three tabs should be visible and switchable at the top -- no sidebar page navigation.

Start with the Chat tab:

- A sidebar with the title "Healthcare Intelligence" and a brief description of the app
- A chat interface where users can have a conversation with HEALTHCARE_AGENT
- Show the agent's responses in the chat as they arrive
- When the agent returns data, display it as a table below the response
- When the agent returns SQL, show it in a collapsible section
- Keep the conversation history visible throughout the session

Apply a clean clinical design throughout the app:

- Main content background: white (#FFFFFF) with light gray panels (#F1F5F9) and subtle drop shadows on cards -- the surface should feel clean and open
- Sidebar: slate blue (#334155) with white text and a teal left border on the active page -- provides clear navigation contrast without being harsh
- Primary color: soft blue (#2563EB) for headings, buttons, and active states; teal (#0D9488) as a secondary accent
- Typography: sentence case throughout -- no all-caps; use medium weight for section headers and regular weight for body text
- Alerts and critical values: green (#16A34A) for healthy/normal, amber (#D97706) for warnings, red (#DC2626) for critical -- used sparingly
- Data tables: white background, alternating light gray rows (#F8FAFC), status pills (small colored badges) rather than full-row highlights
- KPI cards: white background, rounded corners, thin colored left border indicating status, large bold number in dark slate (#1E293B)
- Charts: white background with the soft blue as the primary series color
- Chat input area: white background matching the main content -- seamlessly part of the page, not a floating element
- Overall feel: a clinical operations dashboard -- precise, trustworthy, and easy to read at a glance during rounds or a hospital staff review

5.2 Dashboard Page

Fill in the Dashboard tab of HEALTHCARE_ASSISTANT_APP with the following content.

This page should show live data from HEALTHCARE_HOL_DB.CLINICAL:

TOP ROW - four summary numbers:

- Total Admitted Patients (PATIENTS where STATUS = 'Admitted')
- Bed Occupancy Rate (occupied beds / total beds as percentage)
- Average Length of Stay (from ENCOUNTERS)
- High Readmission Risk Patients (READMISSIONS where RISK_SCORE > 0.7)

MIDDLE ROW - two charts side by side:

- A bar chart of bed occupancy by department, highlighting departments

above 80% occupancy
- A pie chart of encounters by type (Inpatient/Outpatient/Emergency)

BOTTOM ROW - a critical alerts table:

- All patients with RISK_SCORE > 0.7, showing: Patient Name, Department, Primary Diagnosis, Risk Score, Days Since Last Discharge, Readmission Reason
- Sort by risk score descending (highest risk first)
- Include a button to download the table as a CSV

Chart styling:

Use plotly for all charts. Color palette: steel blue (#2563EB) as the primary series color, teal (#0D9488) as the secondary. Use green (#16A34A), amber (#D97706), red (#DC2626) for status-based coloring only. All chart backgrounds should be white (FFFFFF) with light gray grid lines. No dark backgrounds on any chart. Position the legend below each chart (orientation="h", y=-0.3) to prevent overlap with chart titles. Add a top margin (t=50) on all charts so the title never crowds the plot area.

5.3 Optimization Page

Fill in the Optimization tab of HEALTHCARE_ASSISTANT_APP with the following content.

Title: "Clinical Intelligence"

Subtitle: "Optimize bed allocation and forecast readmission risk using AI and machine learning on your live clinical data."

At the top of the tab, add a mode selector using radio buttons:

- "Bed Optimizer"
- "Readmission Risk Forecaster"

MODE 1: Bed Optimizer

Show the following controls in a row above the results:

- A multi-select for Department (options from DEPARTMENTS table, plus "All")
- A slider for Occupancy Alert Threshold: 60% to 95% (default 80%)
- A toggle: "Include patients nearing discharge (LOS > 5 days)"
- A steel blue "Run Optimizer" button

When clicked:

1. Pulls current bed status from BEDS and department occupancy from ENCOUNTERS
2. Finds departments above the selected occupancy threshold
3. If the discharge toggle is on, flags patients with LENGTH_OF_STAY > 5 as near-discharge candidates
4. Uses linear programming (scipy) to suggest moves that balance occupancy while keeping patients in the correct bed type (ICU stays ICU, Pediatric stays Pediatric, etc.)

Display results as:

- Three KPI cards: Departments Over Threshold, Beds That Can Be Freed, Patients Flagged for Discharge Review
- A plotly grouped bar chart showing before/after occupancy by department, with a horizontal reference line at the selected threshold
- A recommended actions table: Patient ID, From Department, To Department, Bed Type, Reason -- sorted by urgency
- Apply the app styling: white background, clinical blue/teal palette, plotly white chart backgrounds

MODE 2: Readmission Risk Forecaster

Show the following controls:

- A dropdown for Department (from DEPARTMENTS table)
- Radio buttons for Forecast Horizon: 7 days / 14 days / 30 days
- A steel blue "Generate Forecast" button

When clicked:

1. Pull the daily count of high-risk patients ($RISK_SCORE > 0.7$) from READMISSIONS joined with ENCOUNTERS, grouped by department and date
2. Use SNOWFLAKE.ML.FORECAST to predict the number of high-risk patients expected in that department over the selected horizon
3. Retrieve forecast values with upper and lower confidence bounds

Display results as:

- A headline KPI card: "Patients at High Readmission Risk (Forecasted)" for the selected horizon period
- A plotly line chart with:
 - Historical high-risk patient count (solid blue line, #2563EB)
 - Forecasted count (dotted line with light blue confidence band)
 - X-axis: dates, Y-axis: patient count
 - Legend below the chart, white background
- A risk summary table: Date, Forecasted High-Risk Count, vs Current Average, Change (% above/below baseline)
- A small note: "Forecast based on [N] days of historical data. Confidence intervals widen with shorter history."

Apply the same styling as the rest of the app throughout:
white background, soft blue primary (#2563EB), teal accent (#0D9488),
plotly white chart backgrounds, sentence case, no all-caps.

Add scipy to the app's dependencies.